

HOJA DE CÓDIGO PURO Y SOLUCIONES DE EXAMEN — TÉCNICAS DIGITALES II

Código Fuente Completo, Bare Metal vs CMSIS y Resoluciones Directas (STM32F103)

UTN-FRC — Cátedra de Técnicas Digitales II (Año 2026)

DOCUMENTO DE CONSULTA DE CÓDIGO DE EXAMEN PRACTICO — Soluciones completas listas para completar el cuestionario de examen práctico.

1. Soluciones del Cuestionario de Examen Práctico (Preguntas 1 a 9)

Pregunta 1: Parpadeo de LED PC13 con Loop For (Bare Metal Puro)

```
1 #define RCC_BASE      0x40021000UL
2 #define GPIOC_BASE    0x40011000UL
3 #define RCC_APB2ENR    (*(volatile uint32_t *) (RCC_BASE + 0x18))
4 #define GPIOC_CRH      (*(volatile uint32_t *) (GPIOC_BASE + 0x04))
5 #define GPIOC_ODR      (*(volatile uint32_t *) (GPIOC_BASE + 0x0C))
6 #define RCC_IOPCEN     (1 << 4)
7 #define GPIOC13        (1UL << 13)
8
9 int main(void) {
10     /* ===== Configuración ===== */
11     RCC_APB2ENR |= RCC_IOPCEN;           // Habilitar reloj GPIOC
12     GPIOC_CRH &= ~(0xF << 20);          // Limpiar bits de PC13
13     GPIOC_CRH |= (0x2 << 20);            // MODE13=10 (2MHz), CNF13=00 (Push-Pull)
14     /* ===== */
15     while (1) {
16         /* ===== parpadeo del LED ===== */
17         GPIOC_ODR ^= GPIOC13;             // Conmutar PC13
18         for (volatile int i = 0; i < 500000; i++); // Retardo por software
19         /* ===== */
20     }
21 }
```

Pregunta 2: Parpadeo de LED PC13 con Loop For (CMSIS)

```
1 #include "stm32f1xx.h"
2
3 int main(void) {
4     /* ===== Configuración ===== */
5     RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; // Habilitar reloj GPIOC
6     GPIOC->CRH &= ~GPIO_CRH_CNF13_Msk;   // CNF13 = 00 (Push-Pull)
7     GPIOC->CRH |= GPIO_CRH_MODE13_1;     // MODE13 = 10 (Output 2MHz)
8     /* ===== */
9     while (1) {
10         /* ===== parpadeo del LED ===== */
11         GPIOC->ODR ^= GPIO_ODR_ODR13;    // Conmutar PC13
12         for (volatile int i = 0; i < 500000; i++); // Retardo por software
13         /* ===== */
14     }
15 }
```

Pregunta 3: EXTI0 en PA0 con Pull-Down conmuta PB13 (Bare Metal Puro)

```
1 #define RCC_APB2ENR    (*(volatile uint32_t *) (0x40021000 + 0x18))
2 #define GPIOA_CRL      (*(volatile uint32_t *) (0x40010800 + 0x00))
3 #define GPIOA_ODR      (*(volatile uint32_t *) (0x40010800 + 0x0C))
4 #define GPIOB_ODR      (*(volatile uint32_t *) (0x40010C00 + 0x0C))
5 #define AFIO_EXTICR1   (*(volatile uint32_t *) (0x40010000 + 0x08))
6 #define EXTI_IMR       (*(volatile uint32_t *) (0x40010400 + 0x00))
7 #define EXTI_RTSR      (*(volatile uint32_t *) (0x40010400 + 0x08))
8 #define EXTI_FTSR      (*(volatile uint32_t *) (0x40010400 + 0x0C))
9 #define EXTI_PR        (*(volatile uint32_t *) (0x40010400 + 0x14))
10 #define NVIC_ISERO     (*(volatile uint32_t *) (0xE000E100))
11
12 #define RCC_AFIOEN      (1UL << 0)
13 #define RCC_IOPAEN      (1UL << 2)
14 #define GPIOA0          (1UL << 0)
15 #define GPIOB13         (1UL << 13)
16 #define EXTI0_IRQN      6
```

```

17
18 void EXTI0_IRQHandler(void);
19
20 int main(void) {
21     /* ===== Configuración de PA0 y EXTI0 ===== */
22     RCC_APB2ENR |= RCC_IOPAEN | RCC_AFIOEN; // Relojes GPIOA y AFIO
23     GPIOA_CRL &= ~(0xF << 0);
24     GPIOA_CRL |= (0x8 << 0); // PA0 Input Pull-Up/Pull-Down (0x8)
25     GPIOA_ODR &= ~GPIOA0; // ODR0 = 0 -> Pull-Down
26
27     AFIO_EXTICR1 &= ~(0xF << 0); // EXTI0 mapeado a PA0 (0000)
28     EXTI_IMR |= (1UL << 0); // Desenmascarar EXTI0
29     EXTI_RTSR |= (1UL << 0); // Flanco Ascendente
30     NVIC_ISERO |= (1UL << EXTI0_IRQN); // Habilitar EXTI0 en NVIC (bit 6)
31     /* ===== */
32     while (1) {
33         /* programa principal */
34     }
35 }
36
37 void EXTI0_IRQHandler(void) {
38     /* ===== Manejador de interrupción ===== */
39     if (EXTI_PR & (1UL << 0)) {
40         EXTI_PR |= (1UL << 0); // Limpiar flag escribiendo UN 1
41         GPIOB_ODR ^= GPIOB13; // Conmutar LED PB13
42     }
43     /* ===== */
44 }

```

Pregunta 4: EXTI0 en PA0 con Pull-Down conmuta PB13 (CMSIS)

```

1 #include "stm32f1xx.h"
2 #include <stdint.h>
3
4 void EXTI0_IRQHandler(void);
5
6 int main(void) {
7     /* ===== Configuración de PA0 y EXTI0 ===== */
8     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN;
9
10    // PA0 como Entrada Pull-Down
11    GPIOA->CRL &= ~(0xF << 0);
12    GPIOA->CRL |= (0x8 << 0);
13    GPIOA->ODR &= ~(1 << 0); // Pull-Down (ODR=0)
14
15    // PB13 ya configurado como salida
16
17    // EXTI0 -> PA0
18    AFIO->EXTICR[0] &= ~(0xF << 0);
19    EXTI->IMR |= EXTI_IMR_MR0;
20    EXTI->RTSR |= EXTI_RTSTR_TR0; // Flanco ascendente
21
22    NVIC_SetPriority(EXTI0_IRQn, 2);
23    NVIC_EnableIRQ(EXTI0_IRQn);
24    /* ===== */
25    while (1) {
26        /* programa principal */
27    }
28 }
29
30 void EXTI0_IRQHandler(void) {
31     /* ===== Manejador de interrupción ===== */
32     if (EXTI->PR & EXTI_PR_PRO) {
33         EXTI->PR |= EXTI_PR_PRO; // Limpiar flag escribiendo 1
34         GPIOB->ODR ^= (1 << 13); // Toggle PB13
35     }
36     /* ===== */
37 }

```

Pregunta 5: Base de Tiempo 1ms y Delay con SysTick @ 72MHz (Bare Metal Puro)

```

1 #define SysTick_CTRL (*(volatile uint32_t *) (0xE000E010))
2 #define SysTick_LOAD (*(volatile uint32_t *) (0xE000E014))
3 #define SysTick_VAL (*(volatile uint32_t *) (0xE000E018))
4 #define SysTick_CTRL_ENABLE (1 << 0)
5 #define SysTick_CTRL_TICKINT (1 << 1)
6 #define SysTick_CTRL_CLKSOURCE (1 << 2)
7
8 volatile uint32_t ticks_ms = 0;
9
10 void systick_init_ms(void) {
11     /* ===== Configuración ===== */
12     SysTick_LOAD = 71999; // (72MHz * 0.001s) - 1
13     SysTick_VAL = 0;
14     SysTick_CTRL = SysTick_CTRL_ENABLE | SysTick_CTRL_TICKINT | SysTick_CTRL_CLKSOURCE;
15     /* ===== */
16 }
17
18 void SysTick_Handler(void) {

```

```

19  /* ===== */
20  ticks_ms++;                // Incrementa milisegundos
21  /* ===== */
22  }
23
24  void delay_ms(int time) {
25  /* ===== */
26  uint32_t start = ticks_ms;
27  while ((ticks_ms - start) < (uint32_t)time);
28  /* ===== */
29  }

```

Pregunta 6: Base de Tiempo 1ms y Delay con SysTick (CMSIS)

```

1  #include "stm32f1xx.h"
2
3  volatile uint32_t ticks_ms = 0;
4
5  void systick_init_ms(void) {
6  /* ===== Configuracion ===== */
7  SysTick_Config(SystemCoreClock / 1000); // Configura 1ms automaticamente a 72MHz
8  /* ===== */
9  }
10
11 void SysTick_Handler(void) {
12 /* ===== */
13 ticks_ms++;
14 /* ===== */
15 }
16
17 void delay_ms(int time) {
18 /* ===== */
19 uint32_t start = ticks_ms;
20 while ((ticks_ms - start) < (uint32_t)time);
21 /* ===== */
22 }

```

Pregunta 7: Lectura ADC1 Canal 2 (PA2) cada 500ms (CMSIS)

```

1  #include "stm32f1xx.h"
2  #include <stdint.h>
3
4  void adc_init(void);
5  int adc_read(int canal);
6  void delay_ms(int tiempo);
7
8  int main(void) {
9  uint16_t valor_adc;
10  adc_init();
11  while (1) {
12  /*===== Leer el canal 2 y esperar 500 ms ===== */
13  valor_adc = adc_read(2);
14  delay_ms(500);
15  /*===== */
16  }
17 }
18
19 void adc_init(void) {
20 /*===== Configurar el GPIO y el ADC1 ===== */
21 RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN; // Reloj GPIOA y ADC1
22 RCC->CFGR &= ~RCC_CFGR_ADCPRE;
23 RCC->CFGR |= RCC_CFGR_ADCPRE_DIV6; // ADCCLK = 72MHz / 6 = 12MHz
24
25 GPIOA->CRL &= ~(0xF << 8); // PA2 Analog Input (MODE=00, CNF=00)
26
27 ADC1->SMPR2 &= ~(0x7 << 6); // Canal 2 -> 55.5 ciclos
28 ADC1->SMPR2 |= (0x5 << 6);
29
30 ADC1->CR2 |= ADC_CR2_ADON; // Encender ADC1
31 /*===== */
32 }
33
34 int adc_read(int canal) {
35 /* ===== Lectura ADC ===== */
36 ADC1->SQR3 &= ~ADC_SQR3_SQ1;
37 ADC1->SQR3 |= (canal & 0x1F); // Cargar canal en 1ra conversion
38
39 ADC1->CR2 |= ADC_CR2_ADON | ADC_CR2_SWSTART; // Iniciar conversion
40
41 while (!(ADC1->SR & ADC_SR_EOC)); // Polling del flag EOC
42
43 return (int)(ADC1->DR & 0xFFFF); // Retornar 12 bits de resultado
44 /* ===== */
45 }

```

Pregunta 8: PWM a 1kHz en TIM2_CH3 (PA2) Duty 10 %, 50 %, 90 % (CMSIS)

```
1 #include "stm32f1xx.h"
2 #include <stdint.h>
3
4 void pwm_init(void);
5 void delay_ms(uint32_t ms);
6
7 int main(void) {
8     pwm_init();
9     while (1) {
10         /*===== Duty Cycle de 10%, 50% y 90% cada 1s ===== */
11         TIM2->CCR3 = 99; // 10% de (ARR + 1) = 1000 -> 100 - 1 = 99
12         delay_ms(1000);
13         TIM2->CCR3 = 499; // 50% -> 500 - 1 = 499
14         delay_ms(1000);
15         TIM2->CCR3 = 899; // 90% -> 900 - 1 = 899
16         delay_ms(1000);
17     } /*===== */
18 }
19
20 void pwm_init(void) {
21     /*===== Configurar el GPIOA y el TIM2 ===== */
22     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN;
23     RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
24
25     GPIOA->CRL &= ~(0xF << 8);
26     GPIOA->CRL |= (0xB << 8); // PA2 Alternate Function Push-Pull 50MHz
27
28     TIM2->PSC = 71; // Clock temporizador = 72MHz / 72 = 1MHz
29     TIM2->ARR = 999; // Periodo = 1000 us = 1ms (1 kHz)
30
31     TIM2->CCMR2 &= ~(TIM_CCMR2_OC3M);
32     TIM2->CCMR2 |= (6 << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE; // PWM Mode 1 + Preload
33     TIM2->CCER |= TIM_CCER_CC3E; // Habilitar salida canal 3
34
35     TIM2->CCR3 = 99; // Inicializar Duty en 10%
36     TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN; // Habilitar Preload y Encender Timer
37     TIM2->EGR |= TIM_EGR_UG;
38 } /*===== */
39
40 }
```

Pregunta 9: Comunicación Serie USART1 a 19200bps 8N1 por Polling (CMSIS)

```
1 #include "stm32f1xx.h"
2 #include <stdint.h>
3
4 void usart1_init(void) {
5     /*===== Configurar el USART1 ===== */
6     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN | RCC_APB2ENR_USART1EN;
7
8     GPIOA->CRH &= ~(0xF << 4);
9     GPIOA->CRH |= (0x9 << 4); // PA9 (TX) Alternate Function Push-Pull 2MHz
10
11     GPIOA->CRH &= ~(0xF << 8);
12     GPIOA->CRH |= (0x4 << 8); // PA10 (RX) Input Floating
13
14     // Baudrate = 19200bps @ 72MHz (PCLK2): USARTDIV = 72000000 / (16 * 19200) = 234.375
15     // Mantisa = 234 (0xEA), Fraccion = 0.375 * 16 = 6 (0x6) -> BRR = 0xEA6
16     USART1->BRR = 0xEA6;
17
18     USART1->CR1 &= ~USART_CR1_M; // 8 bits de datos
19     USART1->CR1 &= ~USART_CR1_PCE; // Sin paridad
20     USART1->CR2 &= ~USART_CR2_STOP; // 1 bit de parada
21
22     USART1->CR1 |= USART_CR1_UE | USART_CR1_TE | USART_CR1_RE; // Habilitar USART1, TX y RX
23 } /*===== */
24
25 void usart1_send_char(char dato) {
26     /*===== Transmision de dato ===== */
27     while (!(USART1->SR & USART_SR_TXE)); // Esperar que TXE este en 1
28     USART1->DR = dato;
29 } /*===== */
30
31 char usart1_receive_dato(void) {
32     /*===== Recepcion dato ===== */
33     while (!(USART1->SR & USART_SR_RXNE)); // Esperar que RXNE este en 1
34     return (char)(USART1->DR & 0xFF); // Retornar dato recibido
35 } /*===== */
36
37 }
```